

Robust and Adaptive Traffic Signal Control for Unstructured Driving Scenarios in the Developing World

Ishan Maheshwari^{1,*}

Ujwal Padam Tewari^{2,*}

Chetan Arora³

Abstract—Adaptive traffic signal control (ATSC) systems are key to efficient traffic management in smart cities. While RADAR or inductive loop based systems are regularly used for the task in the developed world, financial and infrastructure constraints precludes use of sophisticated and expensive sensors in the developing countries. RGB cameras are cheap and easy to deploy, making their usage attractive for ATSC. However, unstructured driving scenarios, and the requirement to work 24×7 pose a significant challenge to the camera based systems. In this paper we propose to use traffic tailback length (TTL) for training a Deep Reinforcement Learning (RL) Agent in an ATSC system. We show that TTL can be calculated by simply performing a ‘background subtraction’ on the input video captured from an RGB camera. The background subtraction operation is well studied in computer vision and can be robustly performed in low illumination, and rainy, snowy, or hazy weather. The operation does not require detecting vehicles explicitly, thus making the proposed system applicable in unstructured driving conditions, involving unseen vehicles. We show, in a simulated environment using SUMO, that our technique outperform the state of the art, which uses explicit vehicle detection information as well as speed of the vehicle, in terms of vehicle throughput, queue length, and average delay.

I. INTRODUCTION

Vehicle density in the cities, especially in the developing world, is increasing exponentially. This is both due to the population growth accompanied by increasing urbanization, as well as rise in the car ownership with improving economic prosperity. However, the road infrastructure is often not able to keep pace with this increase due to financial as well as space constraints. In such a situation adaptive traffic signal control becomes the key to achieving maximum utilization of the existing road infrastructure.

A traffic controller is a system designed to regulate traffic at a single or a set of traffic intersections using static or adaptive rules. While the former creates a fixed phase switching routine based upon the historical traffic data, the adaptive controllers measure the traffic density and vehicle delays at each approach, and then generate an optimal green time for various phases so as to maximize the vehicle throughput at the junction. A typical adaptive traffic signal controller (ATSC) may include sensors, such as RADAR, inductive loops, or cameras, and uses conventional or modern artificial intelligence techniques to learn the incoming traffic patterns. While RADAR and inductive loops are costly and difficult to deploy, the cameras are cheap, and non-intrusive. However, the camera based systems are difficult to operate

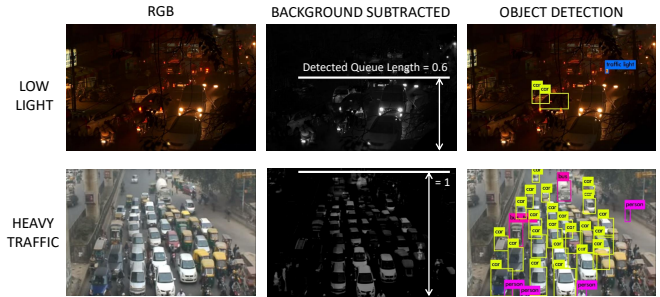


Fig. 1: The figure shows that the traffic tailback length (TTL), which is defined as the distance of the last vehicle standing in a lane at an intersection, can be robustly computed (compared to object detection) even in low illumination and congested self occluding traffic scenarios, by performing ‘background subtraction’ on the input video. Our Reinforcement Learning (RL) agent uses TTL as a cue to learn traffic patterns and regulate phase switching for optimal throughput.

in low illumination or adverse weather conditions, such as rain, snow or haze, when the visibility is low. Besides, these systems often assume lane discipline, and vehicle types to be known a-priori (to detect and track the object and measure the wait time). The assumptions usually do not hold in unstructured traffic in the developing world. The line of sight requirement in camera and RADAR based systems poses additional challenges in the highly congested traffic scenarios due to partial or complete occlusion by other vehicles. In Fig. 1, we show some of the indicative challenging examples for an ATSC system relying on explicit object detection.

Our focus in this paper is on designing a practical and robust adaptive traffic signal control system specifically tailored to the conditions and requirements of the developing countries. Our system uses RGB cameras due to their cost advantage over RADAR, and non-intrusive nature compared to loops which require digging the road. We acknowledge the challenges posed to the camera by low illumination scenario in the night making it difficult to detect vehicles on the road. Here, we make an important observation and contribution that even in the night, the illumination at the road¹ is usually sufficient for a camera to identify the parts of the road occupied by the vehicles. This allows us to observe the traffic tailback at a carriageway. We show in this paper that merely knowing traffic tailback length (called ‘TTL’

¹due to the illumination from vehicle head-lamps as well ambient light from the street-lamps

¹ Delhi Technological University, New Delhi

² Indian Institute of Information Technology Vadodara

³ Indian Institute of Technology Delhi, New Delhi

* Research intern with Chetan Arora during the project.

hereon) is sufficient to learn the incoming traffic pattern, and adaptively adjust the phase switching in an ATSC system to achieve better throughput rates compared to the existing state of the art which makes use of explicit vehicle detection and individual vehicle speeds to decide the phase green timings.

We exploit the advances made in two separate technological domains to design our system. *Background Subtraction* is well studied area in computer vision. In our problem context we learn the background which includes static part of the road, and then compute the foreground representing portions of the road occupied by vehicles. The farthest part of the occupied region is returned as TTL. We use Gaussian Mixture Models [13], [14] to represent the background.

We use advances in artificial intelligence algorithmic techniques, and especially model free reinforcement learning (RL) to train our ATSC system for predicting phase switching time intervals. In the context of our problem, we propose to represent the state of our RL system as a sequence of TTL on all the carriageways, and the corresponding signal phase for last h time instances. Although our RL controller agent can be trained for non-cyclic traffic light switching, we have enforced the traffic lights to be cyclic in nature to ensure the traffic flow does not become chaotic. Therefore, unlike previous works [8], our controller has a much smaller action space - {Switch, Not Switch}. A Switch action sends the traffic light into the next phase of its cycle, whereas a Not Switch action retains the previous phase. As we show in the experiments, this simplifies the training as well as reduces the number of phase cycles an agent needs to observe before it is able to learn the patterns and starts performing optimally.

Contributions: The specific contributions of this work are as follows: (1) We propose to use Traffic Tailback Length (TTL) to learn traffic pattern at an intersection. We show that TTL can be computed by off the shelf, low cost RGB cameras, even in low illumination and adverse weather scenarios. (2) We suggest a reinforcement learning based smart traffic signal controller agent that uses TTL and predicts the phase switching sequence. We make several innovations in design of RL agent, especially for compact and effective state and action representation. (3) We conduct exhaustive experiments using our RL agent in a simulated environment, SUMO [4]. We have tested our agent in (a) changing traffic patterns (b) obstacles in a carriageway, and (c) various rewards such as sign, linear or square function of TTL. We show that in all the scenarios, our agent improves the state of the art in terms of average delay faced by a vehicle as well as overall throughput at the intersection.

II. RELATED WORK

We categorize previous works for adaptive traffic signal control (ATSC) on the basis of information used for designing the RL state space.

One of the major challenges in designing an ATSC system via RL is the choice of state space, and reward function, due to their significant effect on the overall learning process. Touhbi et al. [9] have suggested normalized queue length, obtained by dividing the queue length with lane length,

for generating the state representation. Abdulhai et al. [2] have used the more informative state space by including the queue lengths on the four approaches as well as the elapsed phase time. Liang et al. [5] use a different state space representation based on position and speed of vehicles at an intersection. The action space is defined by selecting every phase's duration in the next cycle. The authors have proposed Double Dueling Deep Q Network (3DQN) algorithm for training their model. Genders and Razavi [3] have proposed to divide the road surface in equal sized cells, and then generate a state representation composed of three vectors. the first vector indicates the presence of a vehicle in each cell, the second records the speed of the vehicle in each cell (0 if vehicle not present) and the third represents the current phase. The representation requires gathering information from expensive sensors and division into cells implicitly assumes lane driving.

Richter et al. [7] have used policy gradient methods for the training. However, they have assumed that all vehicles move at a uniform speed at all times. Further, their controller requires much more elaborate information to create state space, viz cycle duration, phase duration, as well as the information from the neighbouring intersection. Mohamad and Abdulhai [8] have used positions and speeds of the vehicles in the at the intersection as inputs in the form of an image like structure wherein if there is a vehicle on the street, the specific cell corresponding to that part of the street is filled with 1, otherwise 0. Zheng et al. [12] proposed a novel designed called FRAP that models the problem as a competition between pairs of traffic light phases. Through such competition modeling, they claim to achieve invariance to symmetrical cases such as rotation in traffic flow. Zang et al. [11] introduced a novel framework called MetaLight that improved FRAP in traffic signal control by optimizing its model structure and updating paradigm. Xiong et al. [10] use demos of human-operated traffic light signals to train their agent. This avoids the exploration strategy that is required by reinforcement learning agents otherwise.

III. PROPOSED METHOD

A. Simulator

Testing a ATSC system directly in the real-world can be impractical and poses severe risks. To do a proof of concept, we have adopted a simulator based approach. This is inline with the current state of the art, which has also been tested on the simulators. For our simulations, we have used an open-source traffic simulation package 'Simulation of Urban MObility' (SUMO) [4]. However, the frames captured from the simulator do not match the perspective that is expected from the real-world CCTV cameras mounted on the intersections. Therefore, we apply warping and transformation on the simulator frames using OpenCV library [1] to match them with the frames captured from the real camera. This allowed us to test our algorithm on a realistic settings.



Fig. 2: Various stages of our TTL estimation technique.

B. Traffic Tailback Length (TTL)

For separating the foreground from the background, we use a Gaussian Mixture Model (GMM) based Background/Foreground Segmentation Algorithm [13], [14]. Figure 2 shows a result from our background subtraction module on an image taken from a traffic intersection. Note that the background subtraction step helps us to find the foreground pixels, irrespective of the vehicle type, and do not assume any lane driving assumption. The module is also robust against varying illumination and poor visibility due to adverse weather conditions.

To estimate TTL from an incoming frame, we scan the corresponding foreground mask, row by row. For each row we count the number of pixels marked foreground. The row is declared foreground if the count is beyond a threshold. We start counting from the bottom row and move upwards one at a time. The process is stopped when we receive a certain number of consecutive rows which are not foreground. The number of foreground rows is then returned as the estimated TTL. Figure 2 shows an example result.

It may be noted that even when we use a simulated environment, we warp and transform the frames generated by the simulator to visually match the frames captured from a real camera on an intersection. We use our background subtraction module on these frames to compute the TTL. These computed TTL values are then used in the RL agent for a fair comparison with other state of the art.

C. Reinforcement Learning

The goal of a reinforcement learning agent is to observe the environment *state* (in our case TTL in each of the 4 carriageways at the intersection), and then learn to predict an *action* (to switch a given phase or not), which maximize the long term expected *reward* (we explore various rewards). We use *Q-learning* algorithm [6] to train our agent.

The *Q-learning* starts by defining an *action-value* for each action in a particular state, denoted as $Q(s, a)$. The action value, $Q(s, a)$, represents the expected reward in the long term if an action a is chosen given an environment state s . The function $Q(s, a)$ is also known as *Q-function*. Formally, the *policy* of an RL agent, $\pi(s, a)$, is defined as the probability of taking an action, a , given a state, s . Naturally, $Q(s, a)$ is dependent upon the policy, which governs what actions will be subsequently taken. Mathematically:

$$Q^\pi(s_t, a_t) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | s_t, a_t],$$

where R_{t+1} is the reward received at time $t + 1$, and γ is known as *discount factor* for the future rewards.

For scenarios closer to real-world, the state space and action space can be extremely large. So function approximation techniques are used to estimate the *Q-function*. When neural networks are used to estimate the *Q-function*, they are known as Deep Q-Networks (DQN) [6].

Initially, *Q-value* for each state-action pair is randomly initialized. As the agent gains experience by exploring various environment states and receiving rewards by choosing a particular action, it updates the *Q-values* by using well known *Bellman Equation* as follows:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)].$$

Here s' is the new state reached after taking action a in state s , and α is the step size analogous to learning rate in the machine learning systems. *TD-target* is defined as the sum of the current reward and the discounted maximum possible *Q-value* in the the next state for any action. The DQN training algorithm works by minimizing the *TD error*, defined as follows:

$$\text{TD Error} = R(s, a) + \gamma \max_{a'} Q(s', a', w) - Q(s, a, w),$$

where w are the parameters of a neural network to estimate the *Q-function*.

Target Network: We use the idea of fixed *Q-targets* [6], where a separate network, called the target network, is used for estimating the target values and after every n steps, the parameters of the DQN are copied to the target network. The updated TD error can be written as follows:

$$\text{TD Error} = R(s, a) + \gamma \max_{a'} Q(s', a', \bar{w}) - Q(s, a, w),$$

where $\bar{w}$ are the weights of target network. Use of target network significantly improves the DQN training.

Experience Replay: While training a neural network for supervised learning, the samples in a mini batch are uncorrelated. In RL, rewards are sequentially correlated that tend to bias a DQN towards focusing on the latest and forget the previous experiences. To address this one creates a *replay buffer* that stores experience tuples while the agent interacts with the environment. For training, a small batch of tuples is randomly sampled from the replay buffer for training the DQN. This method of creating a memory buffer and randomly sampling from it is called *experience replay*.

In detail, each tuple in replay buffer is of the form (s_t, a_t, r_t, s_{t+1}) . Initially, the buffer is empty. The agent takes random actions till the experience buffer is of the size *init_replay_size*. A maximum size limit, *max_replay_size* is also specified for the buffer. Once the maximum size is reached, each new experience is added by removing the oldest entry from the buffer. For our experiments, we use initial size as 350 tuples and maximum size as 8000 tuples.

ϵ -greedy Policy: While training, we use ϵ -greedy policy [6] to ensure our agent explores the state space before converging to optimal policy. We specify an exploration rate, $\epsilon \in [0, 1]$, and at every time step, with probability $1 - \epsilon$, we *exploit* the current knowledge in the form of learnt *Q-function*, and take action based on the best *Q-value*. However,

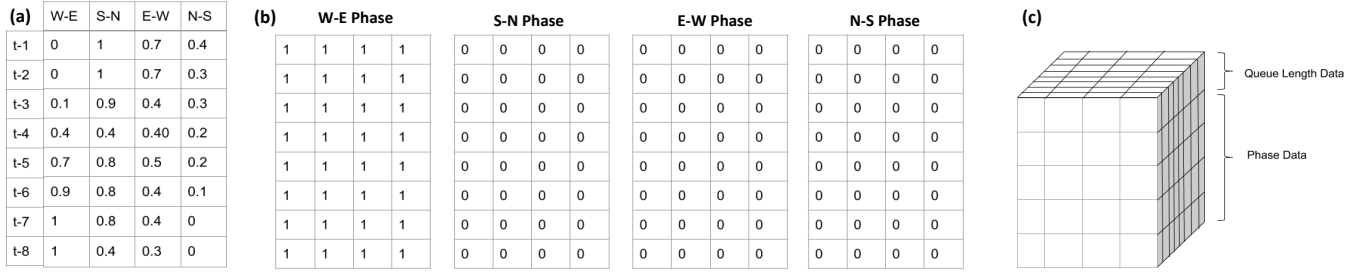


Fig. 3: Figure depicting the steps involved in state construction.

with probability ϵ , we *explore* by choosing a random action. In our experiments, we start with a value of 1 for the ϵ , and uniformly decrease it to 0.4 in 3000 steps. This ensures more exploration when the Q-values are uncertain, and more exploitation when we are confident on the learnt Q-values.

D. Proposed DQN Architecture

1) *State Space*: We record the state information every second, whereas, the granularity of an action in our formulation is of $k = 8$ seconds. In between two action instants, we maintain a history of k time instants for the state representation. The state representation consists of information on queue length ($\in [0, 1]$) in all the carriageways as well as the traffic light phase at the instant. To make it easy to process using standard convolutional neural network (CNN) architecture, we represent the state as a 3D volume of size $5 \times 4 \times k$, where each slice of 5×4 maintains the information at an instant. The first row in a slice contains the queue length in each carriageway. Rows 2 to 5 contains the phase information. Row 2 is filled with all 1 if the signal for WE carriageway is green, or with all 0, if this signal is red. Similarly Row 3, 4, and 5 represents signal phase for SN, EW, and NS carriageways respectively. Though, the information could have been encoded in a single scalar instead of a row of size 4, we use the particular structure to make it a uniform grid which helps in training a CNN architecture. Figure 3 shows the state representation pictorially.

2) *Action Space*: To avoid any kind of chaotic vehicle flow, we follow a cyclic order of traffic light phases. We propose using two actions {Switch, Not Switch} where the Switch action changes the phase to the next phase of the cycle, and Not Switch keeps the phase unchanged. A maximum limit of consecutive Not Switch actions is kept to ensure that the controller does not neglect any lane completely. This limit is set high enough so that it is practically unreachable for a trained controller.

3) *Reward Function*: We observe the queue lengths in various carriageways to find the two scores:

$$\begin{aligned} \mathcal{S}_a &= q_{north} + q_{south} + q_{east} + q_{west}, \text{ and} \\ \mathcal{S}_s &= q_{north}^2 + q_{south}^2 + q_{east}^2 + q_{west}^2, \end{aligned}$$

where q_* denotes the TTL in each of the four carriageways (north, east, south, west). While both the scores serves to bias our agent towards maintaining smaller TTL, $\mathcal{S}_s$ additionally bias the agent towards maintaining relatively shorter TTL in

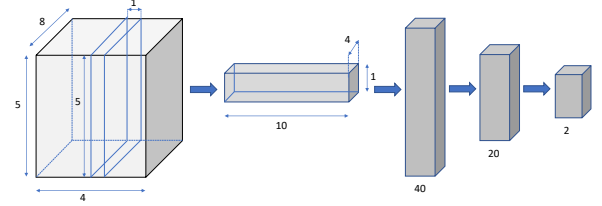


Fig. 4: Architecture of our Deep Q-network



Fig. 5: Comparison of our proposed rewards on a particular traffic pattern. Please see the text for details.

each of the carriageways. This is because at larger values, the square function increases more rapidly than the linear. This biases the agent towards clearing carriageways with larger TTL first. Based upon the scores we propose following reward functions:

$$\begin{aligned} R_1 &= \text{sign}(\mathcal{S}_a^t - \mathcal{S}_a^{t+1}) \\ R_2 &= \mathcal{S}_a^t - \mathcal{S}_a^{t+1} \\ R_3 &= \mathcal{S}_s^t - \mathcal{S}_s^{t+1}. \end{aligned} \quad (1)$$

Here $\mathcal{S}^t$ and $\mathcal{S}^{t+1}$ are the scores at time instants t and $t + 1$ respectively, and $\text{sign}(\cdot)$ is the function which outputs 1 if difference of scores is positive and -1 otherwise. We analyze the performance of our RL agent with all the above reward functions in the experiment section.

4) *Network Architecture*: Our proposed architecture for the DQN is shown in Figure 4. The first layer in our network is a convolutional layer of size 5×1 . The kernel essentially convolves all the information from a carriageway across all the time instants. We learn 10 such filters in the first layer. The output of the layer is thus 10 feature maps of size 1×4 . We flatten the output here to make a single vector of size 40. This vector is then inputted to a fully connected layer of size 20. The third layer is the output layer with 2 neurons, giving the probability of the two actions. Algorithm 1 describes precise steps of the overall training algorithm.

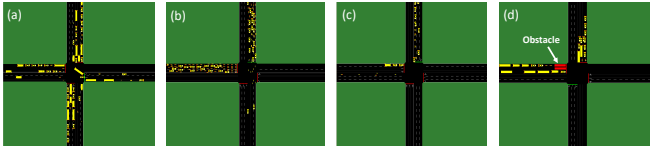


Fig. 6: Scenarios tested on a single intersection. In (a), equal traffic is supplied to all the carriageways. In (b), equal traffic is supplied to two of the carriageways, the other two carriageways are kept empty. In (c), the traffic in the two non-empty carriageways is made unequal. In (d), an obstacle (labeled in red) is introduced in one of the carriageways.

IV. EXPERIMENTS

For comparison with round-robin algorithm, we use a controller that switches traffic lights after every 40 seconds, called Fixed Switching Time 40, or FST-40. We use [8] as representative of state of the art for comparing with our model. Since no implementation is publicly available, we have made our implementation of their algorithm as well. We use state space and reward function as suggested in [8], but kept our action space for easier comparison. We compare the methods on three metrics:

- 1) Average Delay: At every second, we compute the delay faced so far by the vehicles standing in all the carriageways. The average of all the delays is computed as average delay at that instant.
- 2) Queue Length: This is average of the queue lengths (TTL) at all the carriageways, computed every second.
- 3) Vehicles Passed: This is cumulative count of the vehicles which have passed the intersection till that time instant.

A. Comparing Proposed Reward Functions

In the proposed approach, we have explored three rewards to train our ATSC agent (Eq. 1). While R_1 and R_2 give linear reward with the decreasing queue lengths, R_3 gives as per the square of queue lengths. The purpose of R_3 is to penalize increasing longer queues more compared to equivalent increase in the shorter queues. Figure 5 compares the performance of our agent trained with the three rewards. The simulation is tested with traffic in NS carriageway 9 times that of EW carriageway. In all the three cases the agent takes some time to learn the traffic pattern, but after observing the traffic pattern for some time, the performance of the agent is similar on all the three rewards. As we show later in this section, the performance under various proposed rewards differs only when we create an obstacle in one of the carriageways. We have used R_2 as our reward function for subsequent testing with state of the art.

B. Comparison with State of the Art

The controller was compared in a variety of scenarios (Figure 6). In the initial scenario all the four inbound carriageways have equal traffic. This scenario is optimal for the round-robin algorithm. Our controller also learns the round-robin strategy with a shorter switching time and is

able to match the performance of the round-robin algorithm in terms of vehicle throughput, while significantly reducing the cumulative average delay and queue length (Figure 7(a)).

Next, we removed traffic from two carriageways, leaving only two carriageways with equal and non-zero traffic. The controller adapts to this traffic configuration and allocates more time to occupied carriageways than empty carriageways, resulting in superior performance than FST 40. Our method also outperforms state of the art controller [8] which is only able to reduce the cumulative average queue lengths over FST 40 by a narrow margin (Fig. 7(b)).

To further increase complexity, traffic was modified to be unequal in the two carriageways with a ratio of 9:1. This ratio was made to exchange between the 2 carriageways (making it 1:9) at fixed time intervals. Three scenarios were created where this time interval for exchange was 10 minutes, 15 minutes and 30 minutes respectively. The performance was measured on cumulative average delay (Figure 7(c)) and vehicle throughput (Figure 7(d)). When the traffic is exchanged every 30 minutes, the state of the art is able to achieve a better vehicle throughput by a small margin at the cost of a significantly higher delay. This is because the state of the art preferred allocating the green signal almost completely to the denser carriageway. On the other hand, our controller makes sure to allocate green signal to both the carriageways resulting in better queue length management while maintaining a high enough throughput. For the other two scenarios where exchange in traffic happens more frequently at 10 and 15 minutes, our controller outperformed the state of the art and FST 40 in both the metrics. The state of the art performs better than FST 40 for some part of the episode and then worse than FST 40, because it is not able to adapt quickly enough to changing traffic resulting in sub-optimal performance in later stages of the episode.

Next, we stopped exchanging the traffic in the two carriageways and kept the traffic equal in them, while introducing an obstacle on one of the two carriageways, lowering its vehicle speeds. Our controller was compared with FST 40 to check whether it detects the inequality in carriageway vehicle speeds and behaves differently for obstacle carriageway. Figure 8(a) shows that it performs better than FST 40.

To observe the way our proposed controller handled this inequality, we did a carriageway-wise comparison in terms of green time allocated per cycle (Figure 8(c)) and average queue length (Figure 8(b)) observed in obstacle carriageway and non-obstacle (but non-empty) carriageway. We also analyzed the results for our agent trained with rewards R_1 and R_3 to see if the difference in reward function influences the behavior of our controller. Our controller with reward R_2 allocated a significant amount of green time to the non-obstacle carriageway to push the overall throughput higher and to minimize overall queue length by making a trade-off with the average delay time of vehicles on the obstacle carriageway. We also observed that R_1 focuses on queue lengths of both carriageways by allocating more green time to obstacle carriageway than R_2 but fails to find any significant improvements in the queue length of the carriageway with

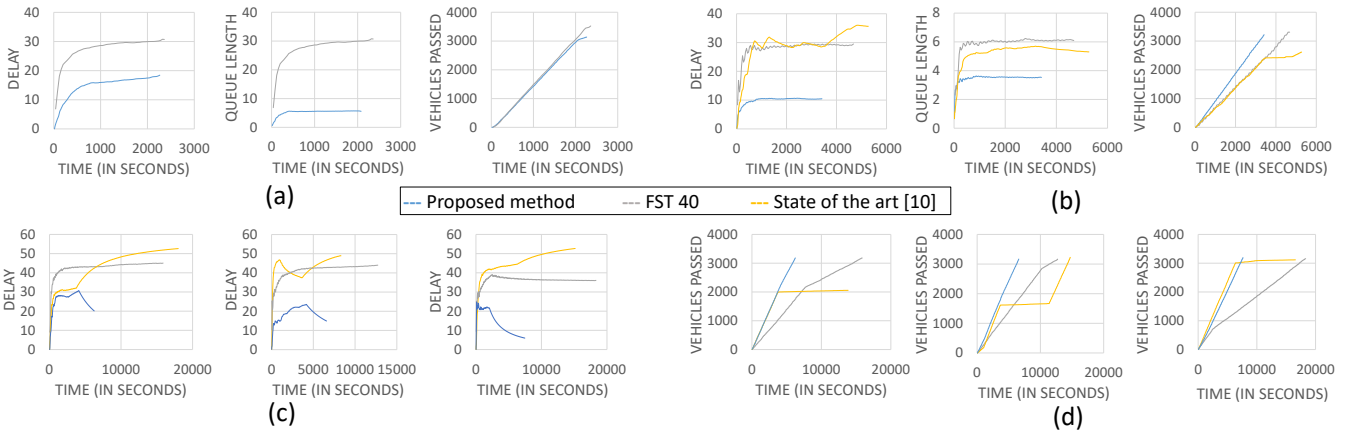


Fig. 7: The graphs depict the comparison of various controllers, including our proposed model, round robin controller having a switching time of 40 seconds (FST 40), and current state of the art model [8].

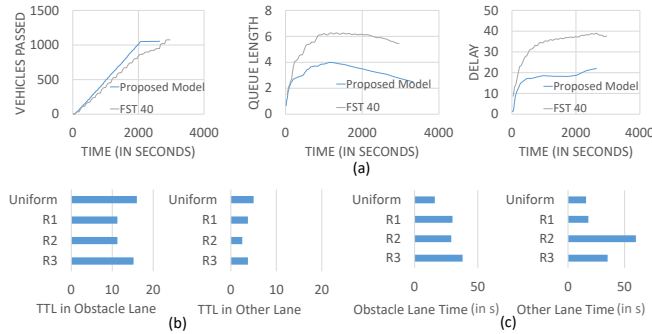


Fig. 8: In (a), the graphs depict the comparison of our proposed model against a round-robin controller (FST-40) in an obstacle scenario. In (b), the graphs depict the behavior of various reward functions.

obstacle because of its lower vehicle speeds. R_3 allocates equal green time to both the carriageways to prevent queue in carriageway with obstacle to grow too long.

V. CONCLUSION

In this paper we have presented a Reinforcement Learning based Adaptive Traffic Signal Controller which uses only the traffic tail back length at each carriageway to learn the incoming traffic patterns and adapt phase switching sequence for optimal throughput. We show that the traffic tail back length can be computed simply and robustly by background subtraction operation on the incoming video frames. The operation is robust to degraded visibility due to adverse weather or night time operation. Further the operation does not assume any lane driving or prior availability of vehicle types. We tested our agent on a variety of commonly occurring scenarios in developing world. In all our experiments the proposed system outperforms the state of the art. We believe our low cost system will make adaptive traffic control practical for use in the developing countries.

REFERENCES

- [1] Open Source Computer Vision Library (OpenCV) Version 3.4.2. <https://github.com/opencv/opencv>, Last Checked: May 2020.
- [2] Baher Abdulhai, Rob Pringle, and Grigoris J. Karakoulas. Reinforcement learning for true adaptive traffic signal control. *Journal of Transportation Engineering*, 129(3):278–285, 2003.
- [3] Wade Genders and Saiedeh Razavi. Using a deep reinforcement learning agent for traffic signal control. *CoRR*, abs/1611.01142, 2016.
- [4] Daniel Krajzewicz, Jakob Erdmann, Michael Behrisch, and Laura Bieker. Recent development and applications of SUMO - Simulation of Urban MObility. *International Journal On Advances in Systems and Measurements*, 5(3&4):128–138, December 2012.
- [5] Xiaoyuan Liang, Xunsheng Du, Guiling Wang, and Zhu Han. Deep reinforcement learning for traffic light control in vehicular networks. *CoRR*, abs/1803.11115, 2018.
- [6] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, February 2015.
- [7] Silvia Richter, Douglas Aberdeen, and Jin Yu. Natural actor-critic for road traffic optimisation. In B. Schölkopf, J. C. Platt, and T. Hoffman, editors, *Advances in Neural Information Processing Systems 19*, pages 1169–1176. MIT Press, 2007.
- [8] S. M. A. Shabestary and B. Abdulhai. Deep learning vs. discrete reinforcement learning for adaptive traffic signal control. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pages 286–293, Nov 2018.
- [9] Saad Touhbi, Momahed Babram, Tri Nguyen-Huu, Nicolas Marilleau, Hbid Moulay Lhassan, Christophe Cambier, and Serge Stinckwich. Adaptive traffic signal control : Exploring reward definition for reinforcement learning. *Procedia Computer Science*, 109:513–520, 06 2017.
- [10] Yuanhao Xiong, Guanjie Zheng, Kai Xu, and Zhenhui Li. Learning traffic signal control from demonstrations. pages 2289–2292, 11 2019.
- [11] Xinshi Zang, Nan Xu, Huaxiu Yao, Guanjie Zheng, Kai Xu, and Zhenhui Li. Metalight: Value-based meta-reinforcement learning for online universal traffic signal control, 2020.
- [12] Guanjie Zheng, Yuanhao Xiong, Xinshi Zang, Jie Feng, Hua Wei, Huichu Zhang, Yong Li, Kai Xu, and Zhenhui Li. Learning phase competition for traffic signal control, 2019.
- [13] Z. Zivkovic. Improved adaptive gaussian mixture model for background subtraction. In *Proceedings of the 17th International Conference on Pattern Recognition, 2004. ICPR 2004.*, volume 2, pages 28–31 Vol.2, Aug 2004.
- [14] Zoran Zivkovic and Ferdinand van der Heijden. Efficient adaptive density estimation per image pixel for the task of background subtraction. *Pattern Recogn. Lett.*, 27(7):773–780, May 2006.